

# CFGS Desarrollo de Aplicaciones Web

Proyecto Intermodular

## PLANFY

*Aplicación de descubrimiento de planes mediante swipe*



**Alumno:** Javier Lugo Benítez

**Tutor docente:** Javier García

**Centro:** IES Los Alcores

Curso 2025-2026

# 1. Índice

Este documento recoge la memoria completa del proyecto intermodular **Planfy**, desarrollado durante el curso 2025-2026 como cierre del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web. La memoria se estructura en seis capítulos numerados que abordan, respectivamente, el análisis del problema, la ejecución técnica, la documentación del sistema y las conclusiones.

## Tabla de contenidos

## 2. Estudio del problema y análisis del sistema

### 2.1. Introducción

Planfy es una aplicación web progresiva (PWA) y multiplataforma orientada al descubrimiento de planes de ocio mediante una interacción tipo «swipe» —similar a la que popularizó Tinder— en la que el usuario desliza una tarjeta hacia la derecha si un plan le resulta atractivo o hacia la izquierda en caso contrario. La aplicación está construida sobre una arquitectura cliente-servidor desacoplada: un frontend desarrollado con Angular 20 e Ionic 8 que se ejecuta tanto en navegador como en dispositivo móvil empaquetado mediante Capacitor, y un backend implementado en Spring Boot 3 con persistencia en PostgreSQL 16, todo orquestado mediante contenedores Docker para facilitar el despliegue.

La aplicación dispone de un catálogo de planes geocalizados en distintas ciudades españolas, organizados por categorías como Cultura, Naturaleza, Gastronomía, Deporte, Ocio nocturno, Arte, Música y Aventura. Cada plan cuenta con información detallada (nombre, descripción, duración, precio, ciudad, categoría, coordenadas e imagen representativa procedente de Wikipedia Commons) que se muestra al usuario en una tarjeta visualmente atractiva. El usuario puede aplicar filtros por ciudad, categoría o precio máximo, realizar búsquedas textuales, gestionar sus planes favoritos y consultar estadísticas personalizadas en su perfil.

### 2.2. Justificación del proyecto

La motivación principal de Planfy nace de la observación de un problema cotidiano: la dificultad de decidir qué hacer en el tiempo libre. Las plataformas existentes (TripAdvisor, Civitatis, Google Maps) presentan listados extensos que requieren un esfuerzo cognitivo elevado para filtrar y comparar opciones. La metáfora del swipe, ampliamente asimilada por los usuarios gracias a aplicaciones como Tinder, Bumble o Stylink, reduce esta carga al presentar una única opción cada vez y permitir una decisión binaria e inmediata.

Adicionalmente, el proyecto integra de forma transversal todos los módulos del ciclo formativo: el modelado de datos en Bases de Datos, la programación orientada a objetos en Programación, el maquetado y comportamiento en Lenguajes de Marcas, los diagramas de análisis en Entornos de Desarrollo, la administración del servidor en Sistemas Informáticos, el desarrollo backend con API REST en Desarrollo Web en Entorno Servidor, el consumo de servicios y validación en Desarrollo Web en Entorno Cliente, el diseño responsive en Diseño de Interfaces Web y la contenedorización en Despliegue de Aplicaciones Web. Por ello, Planfy resulta un caso idóneo para articular y demostrar las competencias adquiridas a lo largo del ciclo.

### 2.3. Objetivos, funciones y rendimientos deseados

Los objetivos generales del sistema se han definido en torno a tres ejes —experiencia de usuario, robustez técnica y aprendizaje del alumno— que guían el resto de decisiones de diseño:

- Construir una aplicación funcional, multiplataforma y responsive que permita descubrir planes de ocio mediante una interfaz intuitiva basada en gestos.
- Implementar una arquitectura de microservicios desacoplada en la que el backend exponga una API REST consumida por el frontend, garantizando la portabilidad y escalabilidad del sistema.

- Aplicar buenas prácticas de seguridad, en particular autenticación basada en JSON Web Tokens, validación de entrada en formularios y manejo controlado de errores en ambos extremos.
- Proporcionar un despliegue reproducible mediante Docker Compose que permita levantar el entorno completo (base de datos, backend y frontend) con un único comando.
- Incorporar funcionalidades complementarias que enriquezcan la experiencia de usuario: modo oscuro automático, sistema de logros, recomendaciones según preferencias previas, deshacer última acción y compartir planes vía Web Share API.

Las funciones concretas del sistema, derivadas de los objetivos anteriores, se enumeran a continuación:

1. Registro y autenticación de usuarios mediante correo electrónico y contraseña, con generación de access token y refresh token JWT.
2. Visualización de planes en formato tarjeta con imagen, categoría, precio, ciudad, descripción y duración.
3. Interacción mediante gestos de swipe (arrastrar la tarjeta) y mediante botones para «me gusta» y «no me gusta», con animaciones suaves de salida.
4. Filtrado por ciudad, categoría, precio máximo y planes gratuitos.
5. Búsqueda textual que detecta automáticamente la ciudad o categoría mencionada y aplica el filtro correspondiente.
6. Modo descubrir, que activa de forma automática planes aleatorios cuando el usuario ha votado todos los disponibles.
7. Listado de favoritos con vista en cuadrícula adaptativa (1 / 2 / 3 / 4 columnas según el tamaño de pantalla) y eliminación con actualización optimista.
8. Pantalla de cuenta con estadísticas personales (categoría favorita, ciudad favorita, total de favoritos, planes gratuitos), selector de tema y sistema de logros desbloqueables.
9. Modal de detalle del plan con imagen ampliada, metadatos y botones de compartir (Web Share API con copia al portapapeles como respaldo) y enlace al mapa.
10. Funcionalidad de deshacer la última acción de swipe.

En cuanto a rendimientos esperados, la aplicación debe responder fluida tanto en navegador de escritorio como en móvil, con un tiempo de carga de la card siguiente inferior a 300 ms gracias a la precarga proactiva (warm-image) y los headers preconnect / dns-prefetch hacia los CDN de imágenes. El backend debe sostener al menos 50 peticiones concurrentes de swipe sin degradación apreciable, cifra adecuada para el ámbito académico del proyecto.

## 2.4. Ciclo de vida del proyecto

El proyecto se ha desarrollado siguiendo un ciclo de vida iterativo e incremental, dividido en cinco fases que se han solapado parcialmente para favorecer la retroalimentación entre análisis e implementación. A continuación se describe cada una de ellas.

### 2.4.1. Fase de análisis

Comprende la identificación del problema, la definición de los objetivos, la elaboración de los requisitos funcionales y no funcionales, así como el diseño preliminar del modelo de datos y de los casos de uso principales. Esta fase culmina con la elaboración de los diagramas Entidad-Relación, de Clases y de Casos de Uso, que se incluyen en el capítulo 3 de esta memoria.

### 2.4.2. Fase de diseño

Definición de la arquitectura del sistema (cliente-servidor con API REST), elección del stack tecnológico, diseño del esquema de base de datos en tercera forma normal, prototipado visual de las pantallas y elaboración del manual de estilo (paleta de colores extraída del logo, tipografía, escala tipográfica). Se decidió en esta fase la separación estricta entre el backend y el frontend, comunicándose únicamente mediante JSON sobre HTTPS.

### 2.4.3. Fase de implementación

Desarrollo del código, dividido a su vez en bloques funcionales que se han abordado en orden de dependencia: primero la base de datos y el modelo JPA, después los servicios de autenticación y el filtro JWT, posteriormente los endpoints REST de planes y favoritos, y finalmente el frontend con sus páginas, servicios e interceptores. La implementación frontend ha seguido la metodología "página por página" (login → dashboard → favoritos → cuenta), validando cada una de forma aislada antes de pasar a la siguiente.

### 2.4.4. Fase de pruebas

Pruebas manuales de aceptación de cada funcionalidad por separado, smoke tests automáticos contra los endpoints REST mediante PowerShell e Invoke-RestMethod, comprobación visual de la responsividad en distintos tamaños de pantalla (móvil 414×844, tablet 768×1024, escritorio 1920×1080), y revisión exhaustiva de errores en consola del navegador. Se han corregido todos los errores HTTP 500 detectados (especialmente los derivados de la expiración del token JWT) reescribiendo el filtro para que devuelva 401 controlado.

### 2.4.5. Fase de despliegue y documentación

Empaquetado de la aplicación en imágenes Docker, definición del fichero docker-compose.yml para orquestar los tres servicios (PostgreSQL, backend, frontend), redacción de los manuales de usuario e instalación, y elaboración de esta memoria.

## 2.5. Recursos

### 2.5.1. Recursos humanos

El proyecto ha sido desarrollado íntegramente por un único alumno (autor de esta memoria), con la supervisión y orientación del tutor docente del módulo. No se ha contado con personal técnico adicional. En el ámbito de los usuarios finales, el sistema está orientado a un perfil de uso individual: cualquier persona mayor de edad con interés en descubrir planes de ocio en ciudades españolas. La autenticación garantiza el aislamiento de datos entre usuarios.

### 2.5.2. Recursos hardware

El desarrollo se ha realizado en un equipo personal estándar y el despliegue se ha llevado al servidor de prácticas del aula. No se ha tenido que adquirir hardware específico para el proyecto, por lo que el coste directo en este apartado es nulo. La siguiente tabla recoge los requisitos mínimos y recomendados para poder trabajar con comodidad en Planfy.

| Recurso | Mínimo | Recomendado |
|---------|--------|-------------|
|---------|--------|-------------|

|                   |                                                 |                                                              |
|-------------------|-------------------------------------------------|--------------------------------------------------------------|
| Procesador        | CPU x64 con 4 núcleos                           | CPU moderna con 8 hilos o más                                |
| Memoria RAM       | 8 GB                                            | 16 GB                                                        |
| Almacenamiento    | 5 GB libres en SSD                              | 10 GB libres en SSD (incluye imágenes Docker y node_modules) |
| Sistema operativo | Windows 10, macOS 12 o Linux moderno            | Windows 11 con WSL 2 / Linux nativo                          |
| Internet          | Banda ancha (descarga única de imágenes Docker) | Conexión estable durante el desarrollo                       |
| <b>Coste</b>      | <b>0 € (equipo personal del alumno)</b>         | <b>0 € (servidor de aula para despliegue)</b>                |

El despliegue de la aplicación se ha realizado sobre el servidor del aula del IES Los Alcores, que ya disponía de Docker instalado. De este modo, el alumno y el profesor pueden acceder a Planfy desde cualquier equipo de la red local del centro sin necesidad de levantar la pila localmente. En un futuro escenario de puesta en producción real, el coste de un VPS modesto rondaría los 4 a 8 euros mensuales, suficiente para soportar tráfico de varios cientos de usuarios al día.

### 2.5.3. Recursos software

Todo el software empleado pertenece al ámbito libre o gratuito para uso académico, por lo que el coste directo del proyecto en licencias es nulo. Los principales componentes son:

| Producto                   | Uso en el proyecto                                                      | Licencia   |
|----------------------------|-------------------------------------------------------------------------|------------|
| Java JDK 21                | Compilación y ejecución del backend Spring Boot                         | GPL/CDDL   |
| Spring Boot 3              | Framework backend, inyección de dependencias y servidor embebido Tomcat | Apache 2.0 |
| Spring Security + JWT 0.11 | Autenticación, autorización y gestión de tokens JWT                     | Apache 2.0 |
| Hibernate / JPA            | Mapeo objeto-relacional sobre PostgreSQL                                | LGPL       |
| PostgreSQL 16              | Base de datos relacional persistente                                    | PostgreSQL |
| Maven                      | Gestor de dependencias y compilación del backend                        | Apache 2.0 |
| Node.js 20 + npm           | Runtime y gestor de paquetes para el frontend                           | MIT        |
| Angular 20                 | Framework SPA del frontend                                              | MIT        |
| Ionic 8                    | Componentes UI mobile-first y plataforma híbrida                        | MIT        |
| Capacitor 7                | Empaquetado en aplicación nativa Android/iOS                            | MIT        |
| Docker Desktop + Compose   | Contenedorización del entorno completo                                  | Apache 2.0 |
| Visual Studio Code         | IDE para frontend, scripts y memoria                                    | MIT        |
| IntelliJ IDEA Community    | IDE para el backend Java/Spring                                         | Apache 2.0 |

|                                 |                                                      |                  |
|---------------------------------|------------------------------------------------------|------------------|
| DBeaver Community               | Cliente gráfico de PostgreSQL para inspección y seed | Apache 2.0       |
| Postman                         | Pruebas manuales de la API REST                      | Freemium         |
| Git + GitHub                    | Control de versiones y alojamiento del código        | GPL/<br>Freemium |
| Trello                          | Gestión de tareas durante la fase de implementación  | Freemium         |
| <b>Coste total de licencias</b> | <b>0 €</b>                                           | —                |

## 3. Ejecución del proyecto

### 3.1. Requisitos funcionales y no funcionales

Como paso previo a la implementación se ha elaborado un catálogo de requisitos siguiendo la nomenclatura RF-XX para los requisitos funcionales y RNF-XX para los no funcionales.

#### 3.1.1. Requisitos funcionales

| Código | Descripción                                                                                                                                |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------|
| RF-01  | El sistema permitirá registrar nuevos usuarios proporcionando nombre, correo electrónico, contraseña y rol.                                |
| RF-02  | El sistema permitirá iniciar sesión a usuarios existentes mediante correo y contraseña, devolviendo un access token y un refresh token.    |
| RF-03  | Los usuarios autenticados podrán solicitar el siguiente plan no votado mediante el endpoint de swipe, con posibilidad de aplicar filtros.  |
| RF-04  | Los usuarios podrán dar «me gusta» o «no me gusta» a un plan; cada voto se persistirá en la base de datos.                                 |
| RF-05  | Los usuarios podrán deshacer su última acción de voto, eliminando el registro correspondiente.                                             |
| RF-06  | Los usuarios podrán consultar el listado de sus planes favoritos en cualquier momento.                                                     |
| RF-07  | El sistema ofrecerá los listados completos de ciudades y categorías para alimentar los selectores del filtro.                              |
| RF-08  | Los usuarios podrán cambiar entre tema claro, oscuro y automático según la preferencia del sistema.                                        |
| RF-09  | La aplicación calculará y mostrará estadísticas individuales: número de favoritos, categoría favorita, ciudad favorita y planes gratuitos. |
| RF-10  | La aplicación incluirá un sistema de logros progresivo desbloqueable conforme el usuario interactúa con planes.                            |
| RF-11  | Los usuarios podrán compartir un plan mediante la Web Share API o, en su defecto, copiando el enlace al portapapeles.                      |
| RF-12  | La aplicación mostrará un tour de bienvenida la primera vez que un usuario accede al panel principal.                                      |

#### 3.1.2. Requisitos no funcionales

| Código | Descripción                                                                                                                      |
|--------|----------------------------------------------------------------------------------------------------------------------------------|
| RNF-01 | Seguridad: las contraseñas se almacenarán hasheadas con BCrypt y los tokens JWT se firmarán mediante HMAC-SHA256.                |
| RNF-02 | Rendimiento: la aplicación responderá a una petición de swipe en menos de 500 ms en condiciones normales de red.                 |
| RNF-03 | Usabilidad: la interfaz se adaptará a pantallas desde 320 px hasta 4K, con experiencia táctil en móvil y de ratón en escritorio. |



|               |                                                                                                                                                           |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RNF-04</b> | Accesibilidad: todos los elementos interactivos dispondrán de aria-label; se respetará prefers-reduced-motion.                                            |
| <b>RNF-05</b> | Mantenibilidad: el código seguirá las convenciones de Angular (servicios standalone) y Spring (capas Controller-Service-Repository).                      |
| <b>RNF-06</b> | Portabilidad: el sistema podrá levantarse en cualquier máquina con Docker Compose mediante un único comando.                                              |
| <b>RNF-07</b> | Robustez: el backend devolverá errores HTTP semánticamente correctos (401 ante token caducado, 403 ante falta de permisos, 404 ante recurso inexistente). |
| <b>RNF-08</b> | Internacionalización futura: los textos están centralizados en plantillas, listos para una posible extracción i18n.                                       |

## 3.2. Diagrama Entidad-Relación

El modelo de datos de Planfy consta de cinco tablas principales relacionadas mediante claves foráneas. La tabla central es planes, que mantiene relaciones N:1 con ciudades y categorías para facilitar la categorización de cada plan. La tabla usuarios se vincula con roles también mediante una relación N:1, y se conecta con planes a través de la tabla intermedia user\_like\_plan, que materializa una relación N:M y registra la dirección del voto (campo liked booleano) y el momento de su creación.

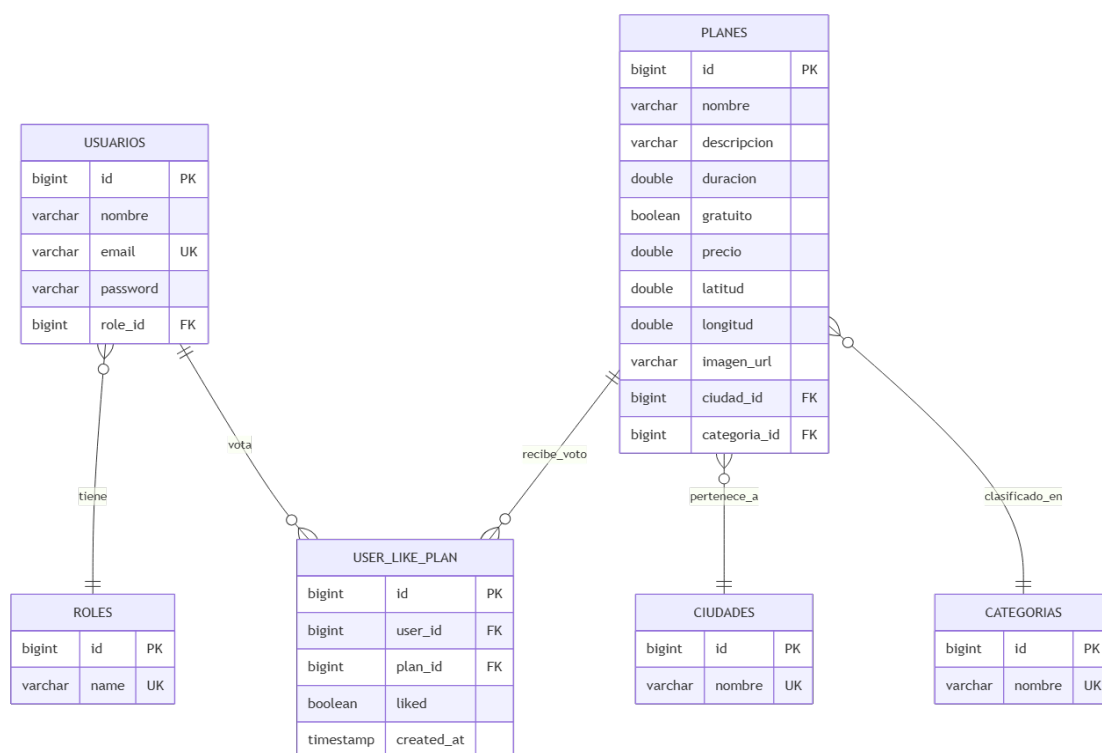


Figura 1. Diagrama Entidad-Relación de Planfy

A continuación se muestra el script DDL simplificado de las tablas principales:

```

CREATE TABLE roles (
  id BIGSERIAL PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL
);
  
```

```
CREATE TABLE usuarios (
  id          BIGSERIAL PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL,
  password    VARCHAR(255) NOT NULL,
  role_id     BIGINT REFERENCES roles(id)
);

CREATE TABLE ciudades ( id BIGSERIAL PK, nombre VARCHAR(80) UNIQUE );
CREATE TABLE categorias ( id BIGSERIAL PK, nombre VARCHAR(80) UNIQUE );

CREATE TABLE planes (
  id          BIGSERIAL PRIMARY KEY,
  nombre      VARCHAR(150) NOT NULL,
  descripcion  VARCHAR(1000),
  duracion    DOUBLE PRECISION,
  gratuito    BOOLEAN,
  precio      DOUBLE PRECISION,
  latitud     DOUBLE PRECISION,
  longitud    DOUBLE PRECISION,
  imagen_url  VARCHAR(500),
  ciudad_id   BIGINT REFERENCES ciudades(id),
  categoria_id BIGINT REFERENCES categorias(id)
);

CREATE TABLE user_like_plan (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT REFERENCES usuarios(id),
  plan_id     BIGINT REFERENCES planes(id),
  liked       BOOLEAN NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (user_id, plan_id)
);
```

### 3.3. Casos de uso

Se han identificado tres actores principales —Usuario anónimo, Usuario autenticado y Administrador— y los siguientes casos de uso que abarcan la totalidad de las funcionalidades del sistema:

- CU-01 Registrarse en la plataforma (actor: Usuario anónimo).
- CU-02 Iniciar sesión (actor: Usuario anónimo).
- CU-03 Ver siguiente plan según preferencias y filtros (Usuario autenticado).
- CU-04 Dar «me gusta» o «no me gusta» a un plan (Usuario autenticado).
- CU-05 Consultar listado de favoritos (Usuario autenticado).
- CU-06 Eliminar un plan de favoritos (Usuario autenticado).
- CU-07 Ver detalles ampliados de un plan (Usuario autenticado).
- CU-08 Compartir un plan (Usuario autenticado).
- CU-09 Cambiar el tema de la aplicación (Usuario autenticado).
- CU-10 Cerrar sesión (Usuario autenticado).
- CU-11 Gestionar el catálogo de planes —alta, modificación, baja— (Administrador, ampliación futura).

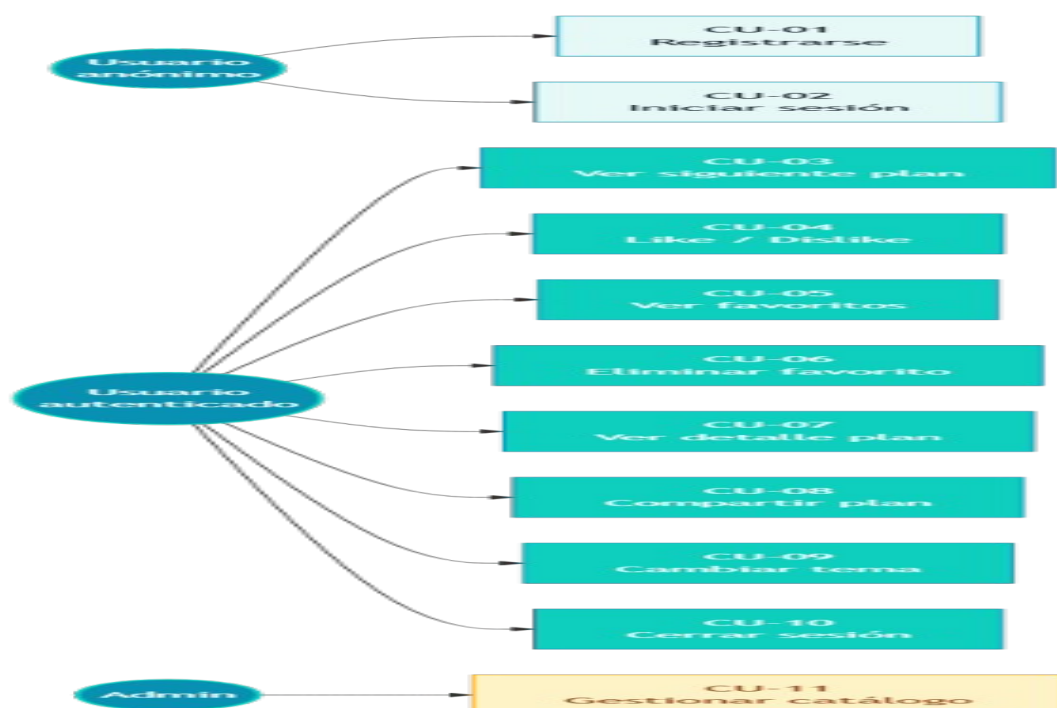


Figura 2. Diagrama de casos de uso

### 3.4. Diagrama de clases

El diagrama de clases del backend refleja la separación clásica en capas Controller-Service-Repository propia de Spring. Las entidades JPA (Plan, Ciudad, Categoría, Usuario, Rol, UserLikePlan) representan las tablas de la base de datos. Los servicios (PlanService, LikeService, AuthService, JwtService) encapsulan la lógica de negocio. Los controladores (PlanController, LikeController, AuthController, MetaController) exponen los endpoints REST. Finalmente, los componentes de seguridad (JwtAuthenticationFilter, SecurityConfig, CustomUserDetailsService) gestionan la autenticación.

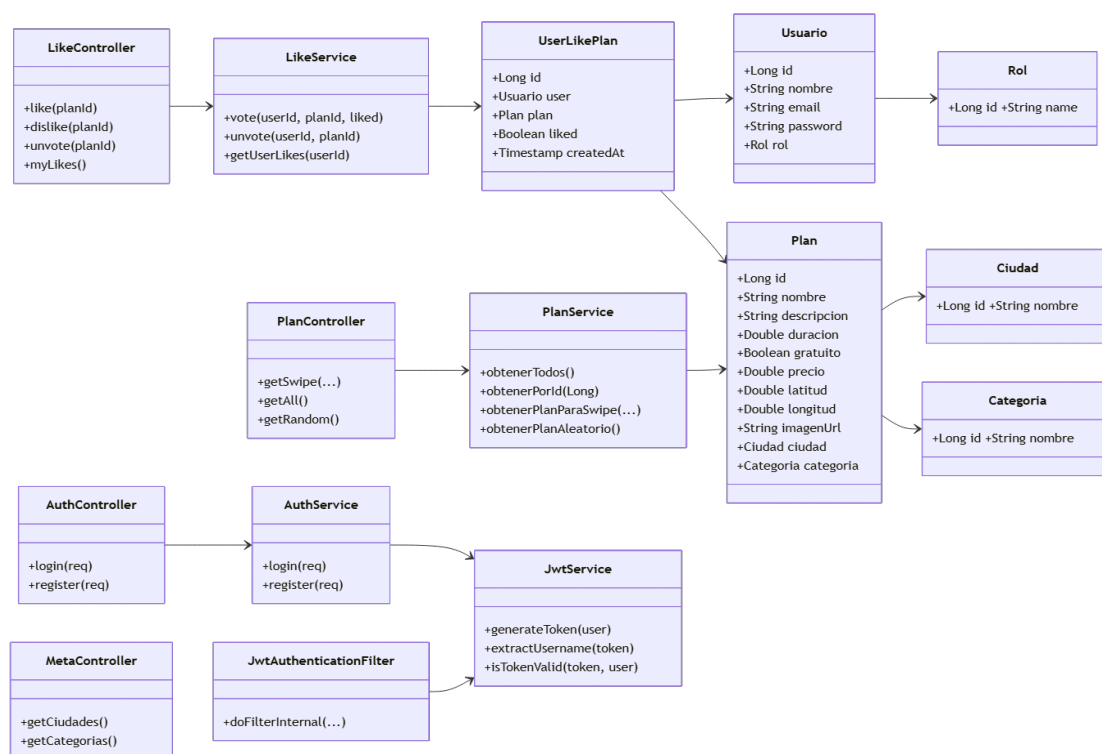


Figura 3. Diagrama de clases del backend

En el frontend, la arquitectura sigue el modelo de servicios inyectables y componentes standalone propios de Angular 17+. Los servicios principales son AuthService, PlanService, MetaService, ImageService, ProgressService y ThemeService. Los componentes de página (LoginPage, DashboardPage, FavoritesPage, AccountPage) consumen estos servicios mediante inyección con la función inject.

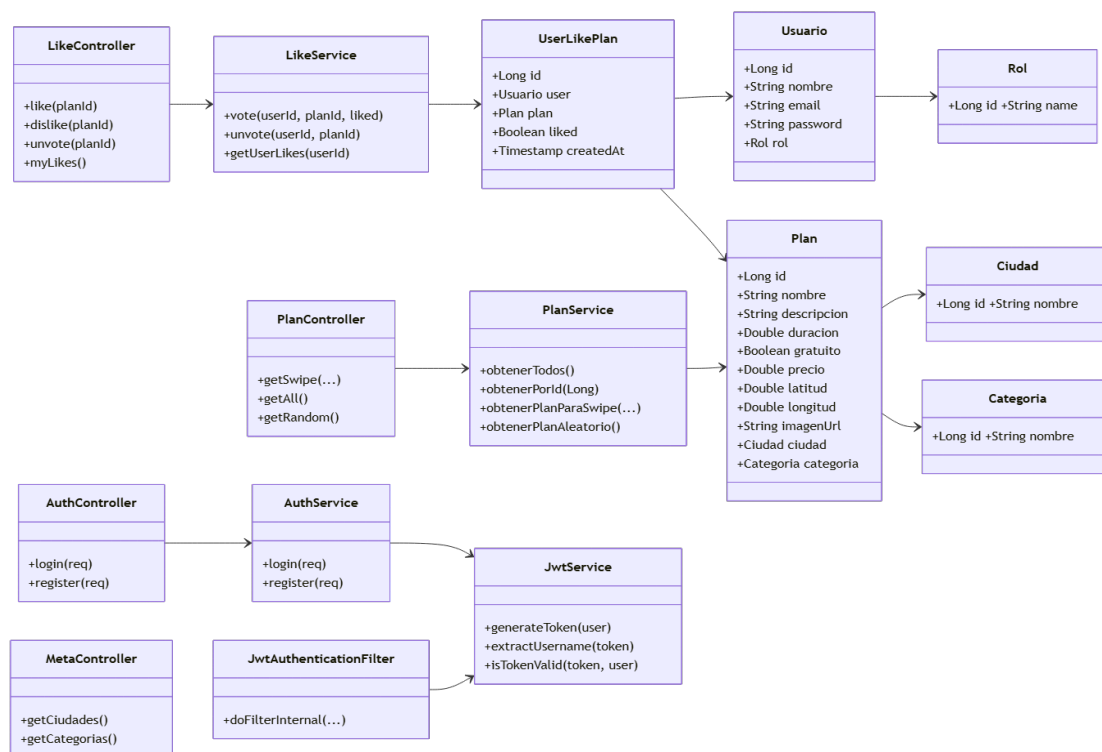


Figura 4. Relación entre páginas y servicios del frontend (mismo modelo conceptual)

## 3.5. Documentación técnica

### 3.5.1. Arquitectura general del sistema

Planfy se asienta sobre una arquitectura de tres capas perfectamente desacopladas que se comunican mediante protocolos estándar (HTTP/JSON entre frontend y backend, JDBC entre backend y base de datos). Esta separación garantiza que cualquiera de las tres capas pueda ser sustituida —por ejemplo, intercambiando Angular por React en el cliente, o PostgreSQL por MySQL en la persistencia— sin afectar al resto del sistema.

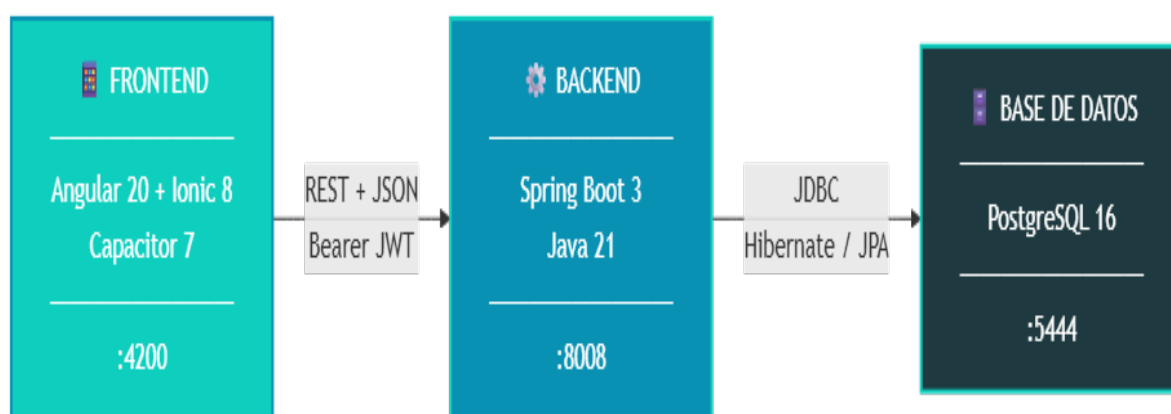


Figura 5. Arquitectura de tres capas desacopladas

### 3.5.2. Base de datos

La persistencia se ha implementado en PostgreSQL 16 ejecutándose en un contenedor Docker. Hibernate genera el esquema automáticamente mediante `spring.jpa.hibernate.ddl-auto=update` y la población inicial se ha realizado mediante el script `datos-prueba.sql`, que inserta 8 ciudades, 8 categorías y 49 planes con sus correspondientes coordenadas y URLs de imagen procedentes de Wikipedia Commons. El campo `imagen_url` se añadió en una iteración posterior una vez que se decidió incorporar fotografías reales en las tarjetas, sustituyendo los gradientes por defecto.

 **PEGAR CAPTURA AQUÍ — 14-dbeaver-tabla-planes.png**

Figura 6. Estructura de la tabla planes en DBeaver. (Pega aquí la captura)

 **PEGAR CAPTURA AQUÍ — 15-dbeaver-select.png**

Figura 7. Datos reales en la tabla planes. (Pega aquí la captura)

### 3.5.3. Backend — Spring Boot

El backend está estructurado en los paquetes habituales del patrón MVC para REST: `controller`, `service`, `repository`, `model`, `dto` y `security`. Cada controlador delega la lógica en un servicio, que a su vez utiliza los repositorios JPA para acceder a la base de datos.

Como ejemplo representativo se incluye a continuación el filtro `JwtAuthenticationFilter`, responsable de validar el token JWT en cada petición y, en caso de detectar un token expirado, devolver un código `401 Unauthorized` con un mensaje claro en JSON en lugar de propagar la excepción y producir un error 500. Esta corrección fue clave durante la fase de pruebas, en la que se observó que la

primera implementación del filtro permitía que la excepción `ExpiredJwtException` burbujease hasta el dispatcher, generando un 500 que el frontend no podía gestionar correctamente.

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Autowired private JwtService jwtService;
    @Autowired private CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse
res, FilterChain chain)
        throws ServletException, IOException {
        String path = req.getServletPath();
        if (path.startsWith("/auth") || path.startsWith("/error"))
{ chain.doFilter(req, res); return; }

        String header = req.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) { chain.doFilter(req,
res); return; }

        String jwt = header.substring(7);
        try {
            String email = jwtService.extractUsername(jwt);
            if (email != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails ud = userDetailsService.loadUserByUsername(email);
                if (jwtService.isTokenValid(jwt, ud.getUsername())) {
                    var auth = new UsernamePasswordAuthenticationToken(ud, null,
ud.getAuthorities());
                    auth.setDetails(new
WebAuthenticationDetailsSource().buildDetails(req));
                    SecurityContextHolder.getContext().setAuthentication(auth);
                }
            }
        } catch (ExpiredJwtException ex) {
            res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            res.setContentType("application/json");
            res.getWriter().write("{\"error\":\"token_expired\"}");
            return;
        } catch (JwtException | IllegalArgumentException ex) {
            res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            res.setContentType("application/json");
            res.getWriter().write("{\"error\":\"invalid_token\"}");
            return;
        }
        chain.doFilter(req, res);
    }
}
```

 **PEGAR CAPTURA AQUI — 16-intellij-backend.png**

*Figura 8. Estructura del proyecto backend en IntelliJ IDEA. (Pega aquí la captura)*

### 3.5.4. Frontend — Angular y Ionic

El frontend se ha desarrollado con Angular 20 e Ionic 8 utilizando exclusivamente componentes standalone, lo que elimina la necesidad de NgModules y simplifica notablemente la organización del código. La estructura sigue las convenciones del CLI de Angular: una carpeta pages para los

componentes de pantalla, una carpeta `services` para los servicios inyectables, una carpeta `interceptors` para interceptores HTTP y una carpeta `guards` para los guardas de ruta.

Como ejemplo se muestra el `authInterceptor`, encargado de inyectar el token JWT en cada petición saliente (excepto en las rutas públicas de autenticación) y de capturar las respuestas 401 para redirigir automáticamente a la pantalla de login. Su correcto registro en `main.ts` fue otro de los puntos críticos detectados durante las pruebas: la primera versión del proyecto registraba el interceptor en un fichero `app.config.ts` que no era importado por `main.ts`, lo que provocaba que ninguna petición autenticada llegase con el header `Authorization`.

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const storage = inject(StorageService);
  const router = inject(Router);
  const PUBLIC = ['/auth/login', '/auth/register'];
  const isPublic = PUBLIC.some(p => req.url.includes(p));
  const token = isPublic ? null : storage.getToken();

  const authReq = token
    ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } })
    : req;

  return next(authReq).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401 && !isPublic) {
        storage.clearAll();
        router.navigate(['/login']);
      }
      return throwError(() => err);
    })
  );
};
```

 [PEGAR CAPTURA AQUÍ — 17-vscode-frontend.png](#)

*Figura 9. Estructura del proyecto frontend en VS Code. (Pega aquí la captura)*

### 3.5.5. Capturas de la aplicación

A continuación se incluyen las capturas más representativas de la interfaz, tomadas tanto en modo claro como en modo oscuro y en distintos tamaños de pantalla.

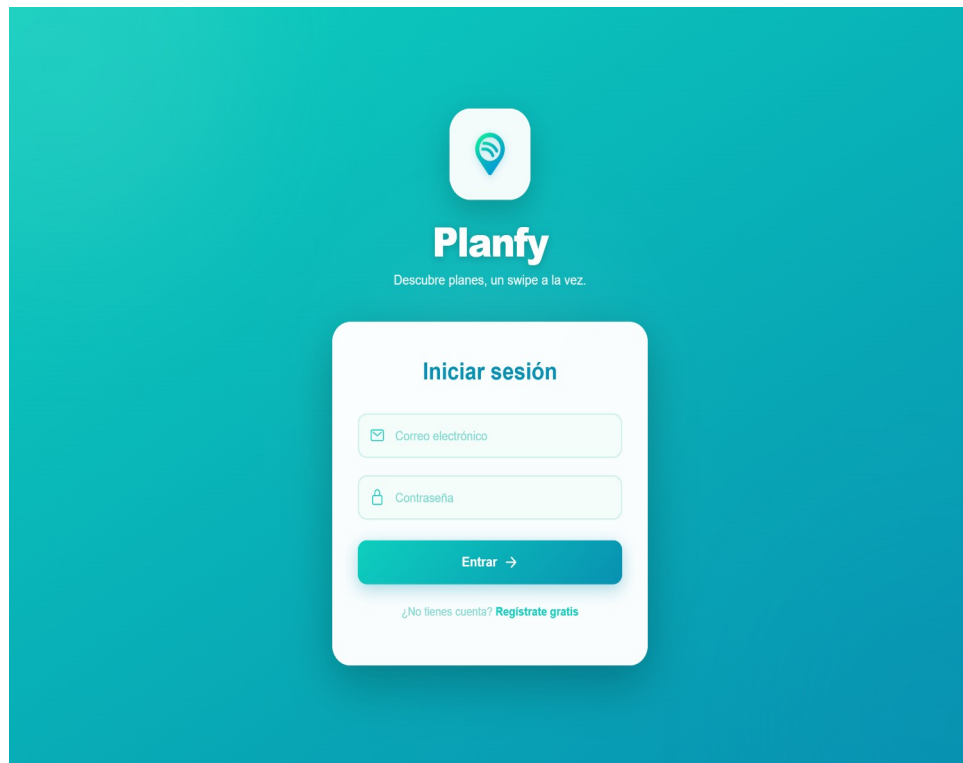


Figura 10. Pantalla de inicio de sesión en modo oscuro

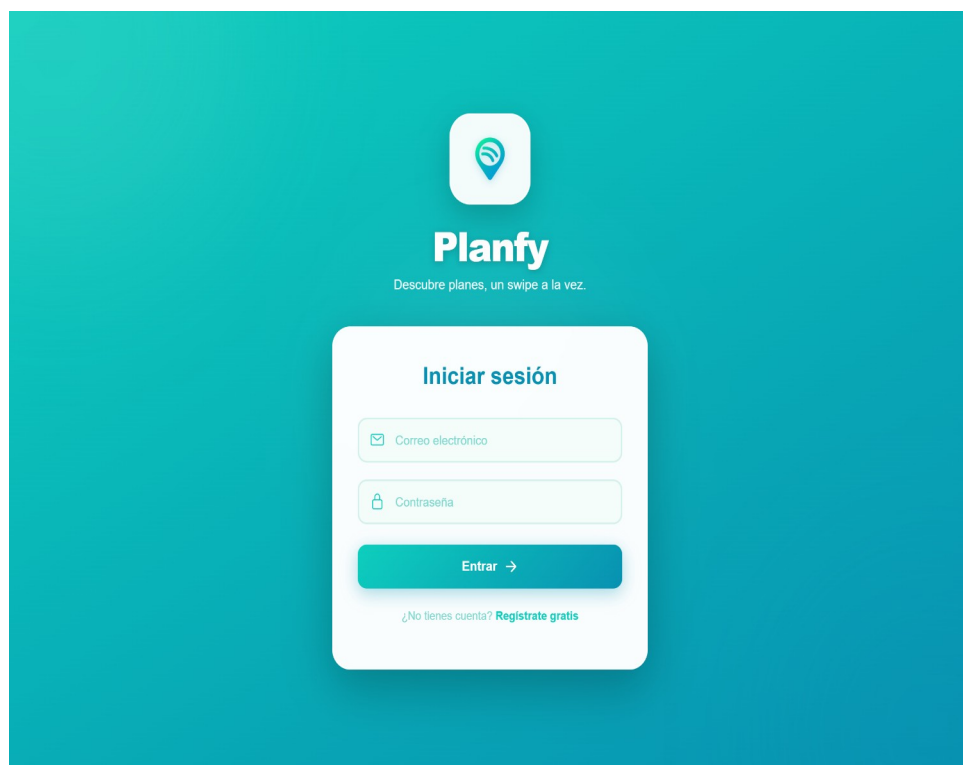


Figura 11. La misma pantalla en modo claro: el sistema de temas funciona



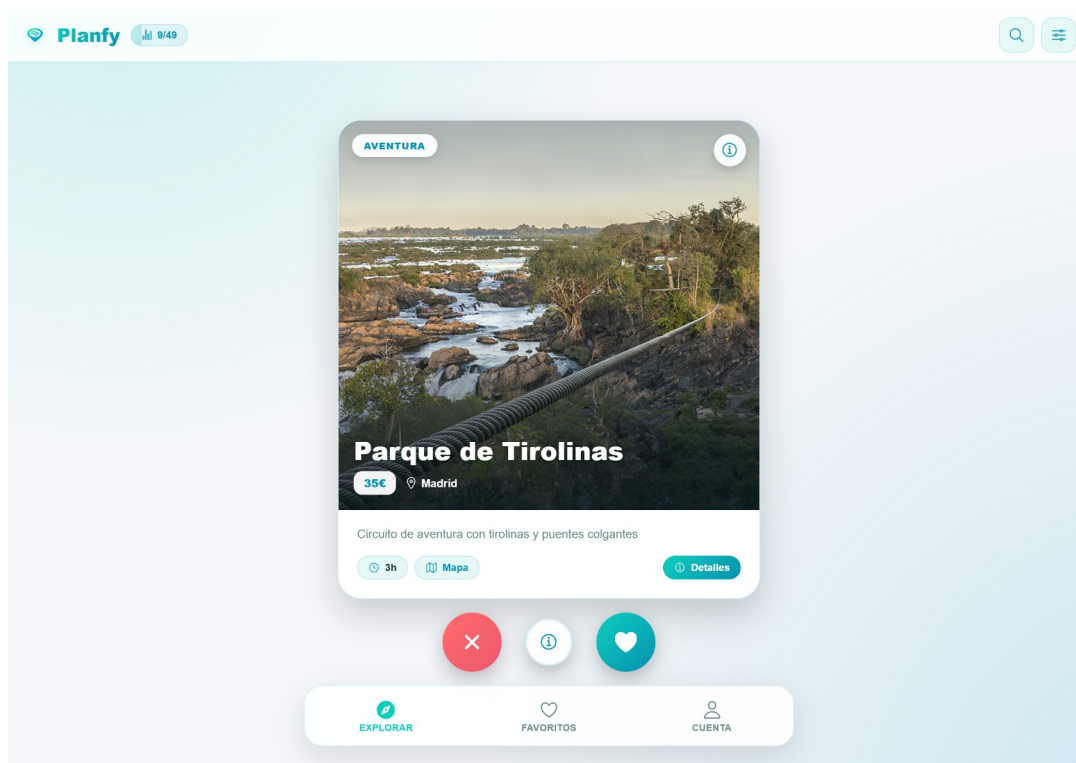


Figura 12. Dashboard principal en escritorio: imagen del plan, badges, descripción y botones de acción

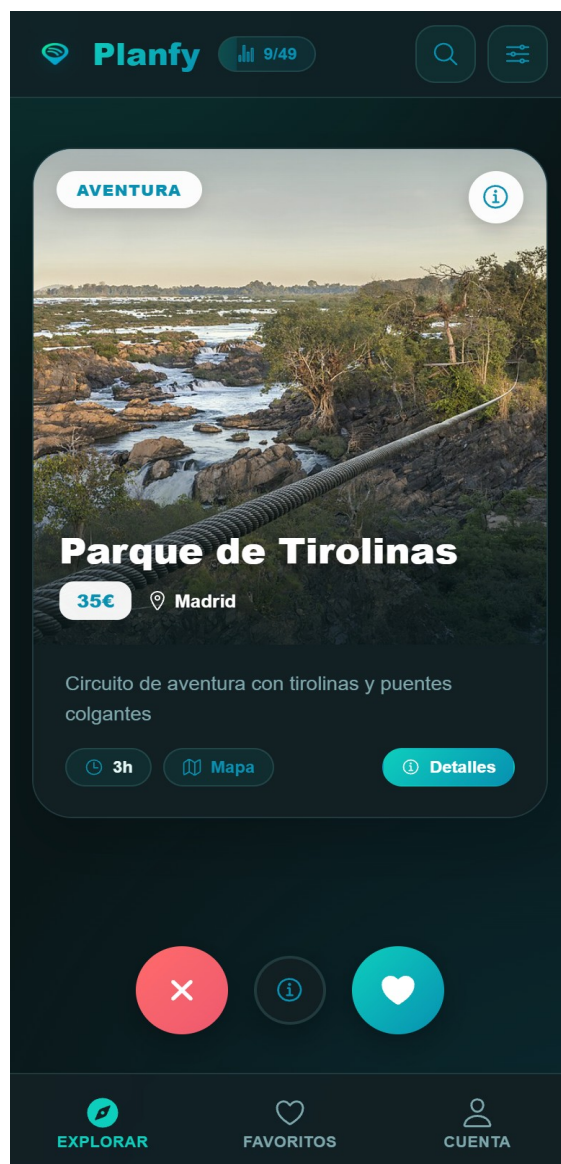


Figura 13. Mismo dashboard en móvil (390 × 844 px) — el layout adapta el ancho de la card y compacta la cabecera

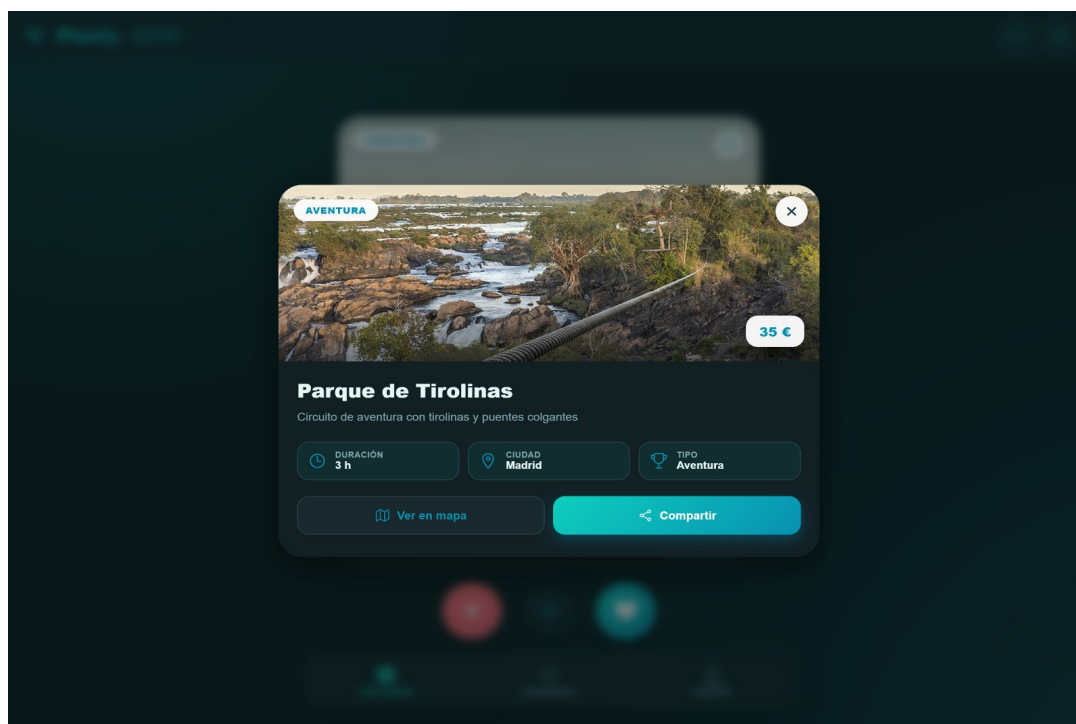


Figura 14. Modal de detalle del plan: imagen hero ampliada, metadatos y acciones (mapa y compartir)

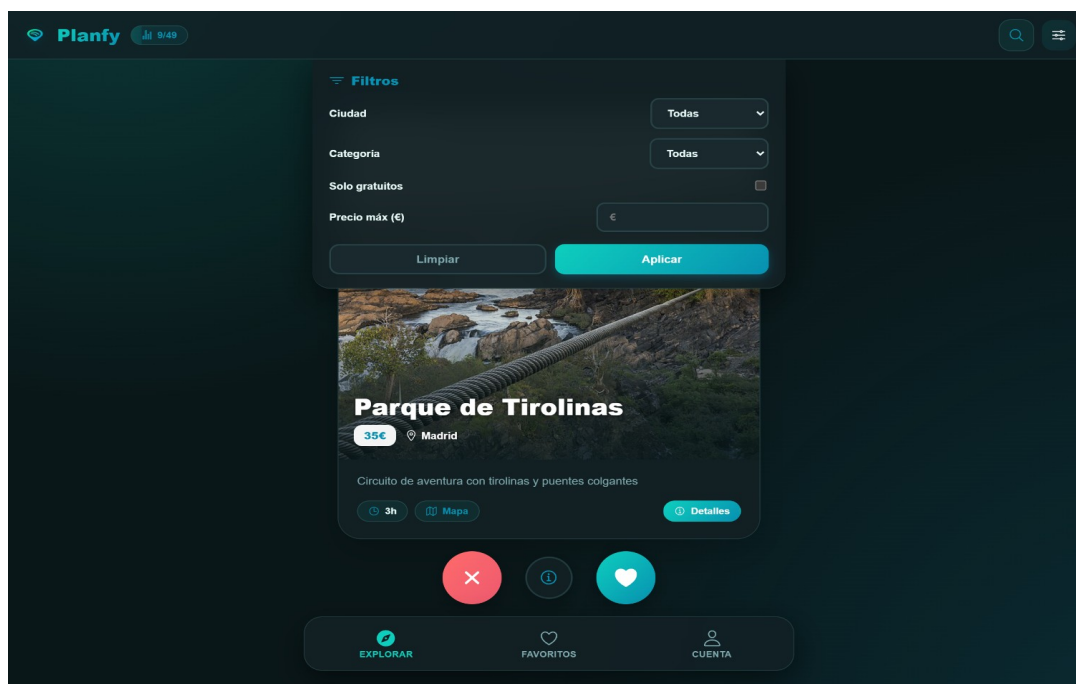


Figura 15. Panel de filtros desplegado con dropdowns dinámicos de ciudad y categoría

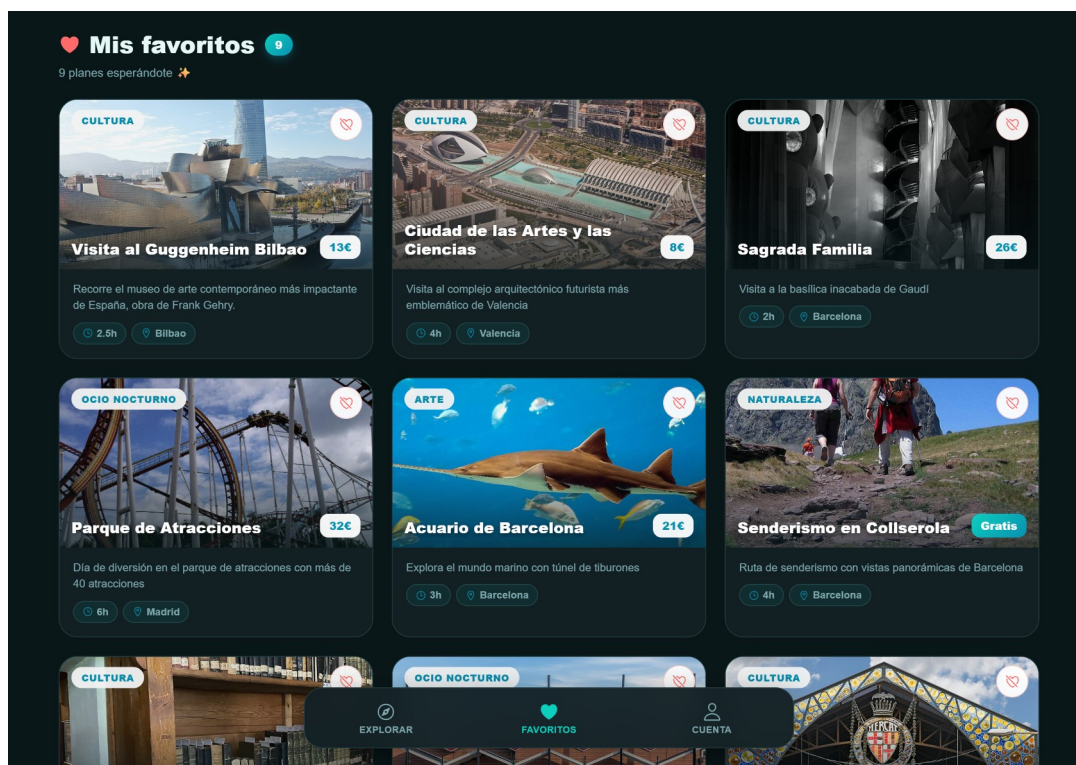


Figura 16. Pantalla de favoritos en escritorio con cuadrícula adaptativa de 3 columnas

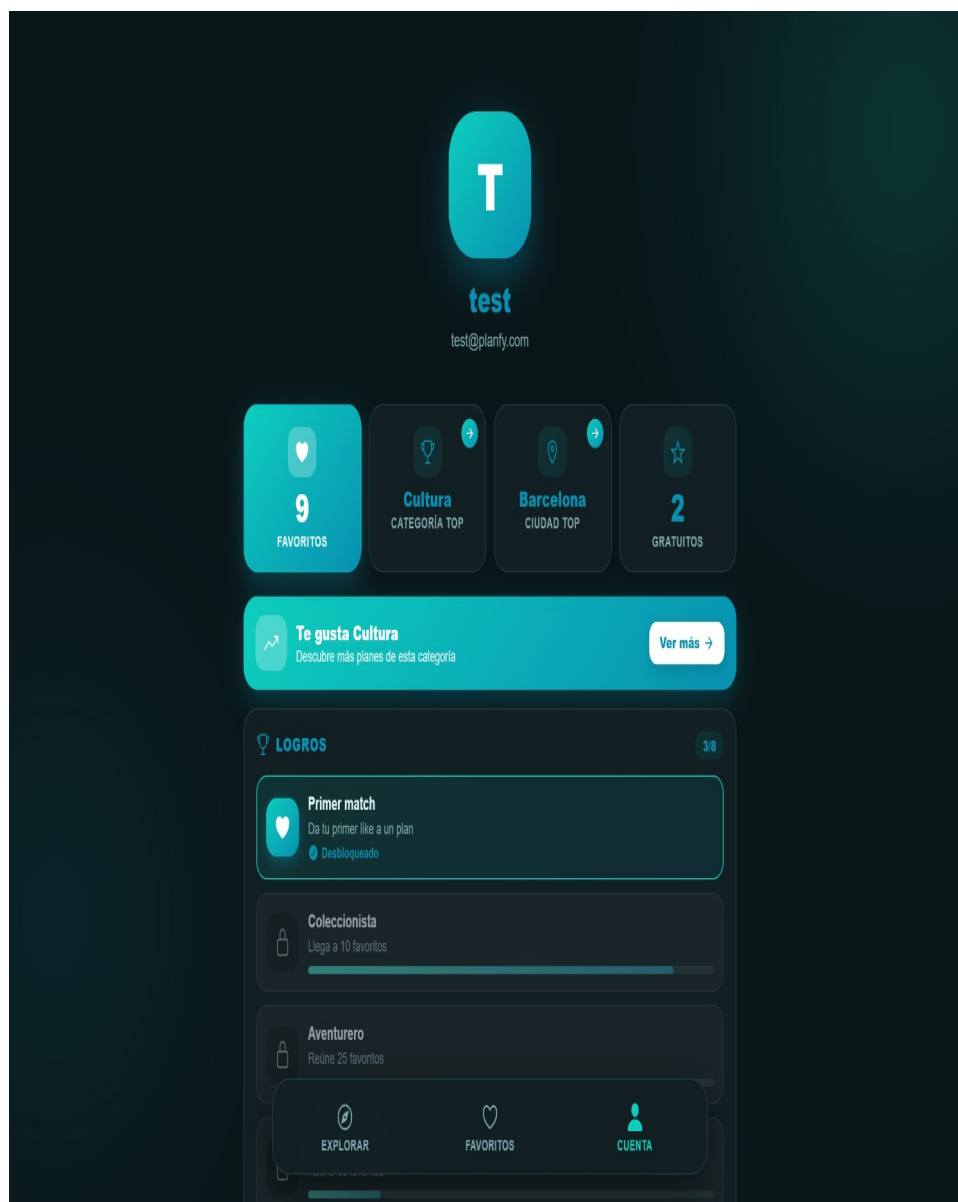


Figura 17. Pantalla de cuenta: avatar, estadísticas, recomendación, selector de tema, logros y acciones

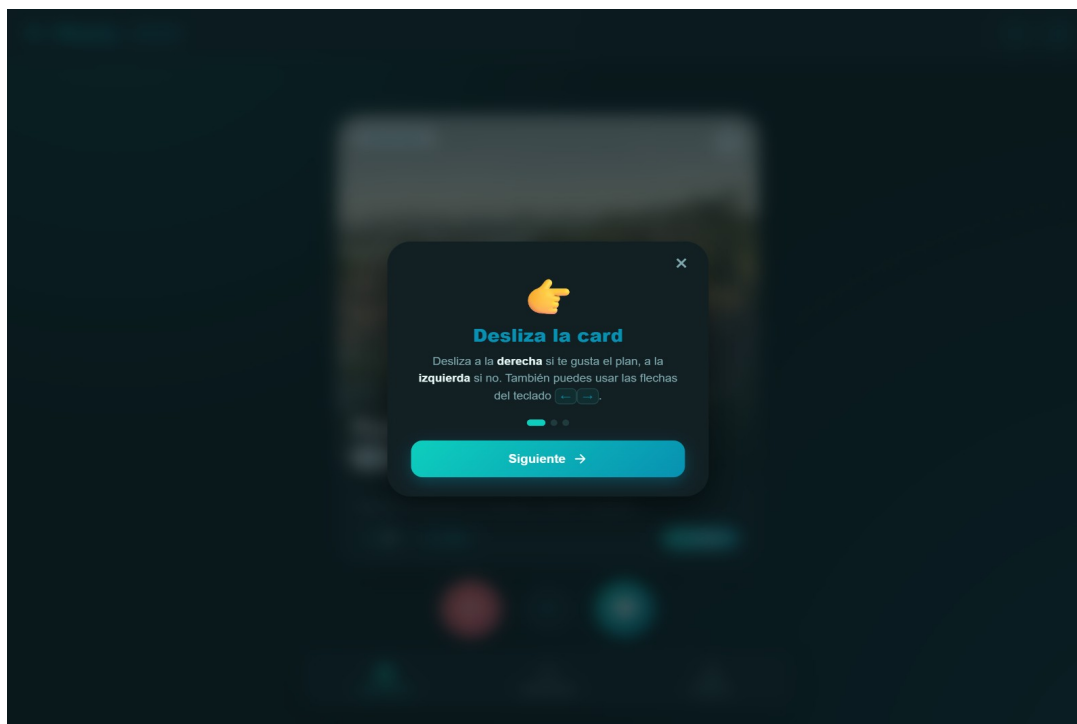


Figura 18. Tour de bienvenida en su primer paso (solo se muestra la primera vez tras registrarse)

### 3.6. Enlaces al repositorio y descarga

El código fuente íntegro del proyecto se aloja en GitHub bajo licencia MIT. Cualquier persona con la dirección puede clonar el repositorio, levantar el entorno y comenzar a probar la aplicación en menos de cinco minutos siguiendo el manual de instalación incluido en el capítulo 4.

Repositorio principal:

Descarga directa (ZIP):

## 4. Documentación del sistema

### 4.1. Manual de instalación, despliegue y configuración

Planfy ha sido diseñado desde el primer momento para ser totalmente reproducible mediante Docker Compose. Esta sección recoge los pasos detallados para levantar el entorno completo en una máquina nueva.

Existen dos modos de uso de la aplicación: ejecución local en el equipo del alumno (útil durante el desarrollo) y acceso al despliegue centralizado en el servidor del aula del IES Los Alcores, que es el modo en el que el tutor podrá probar la aplicación durante la corrección.

#### 4.1.1. Requisitos previos

- Docker Desktop instalado (Windows 10/11 con WSL 2, macOS 13+ o cualquier distribución Linux moderna).
- Git instalado para clonar el repositorio.
- Mínimo 4 GB de RAM libres y 5 GB de espacio en disco.
- Conexión a Internet la primera vez (para descargar las imágenes Docker base de PostgreSQL y Eclipse Temurin).
- Puertos 4200, 5444 y 8008 disponibles en localhost.

#### 4.1.2. Pasos de instalación

11. Clonar el repositorio: `git clone https://github.com/JLugoBenitez/Planfy.git` && `cd Planfy`
12. Verificar que Docker Desktop está en ejecución y muestra el icono verde "Engine running".
13. Levantar todos los servicios con un único comando: `docker compose up -d`
14. Esperar aproximadamente 60 segundos hasta que el contenedor `planfy_backend` esté en estado `healthy`.
15. Sembrar la base de datos con los datos de prueba: `docker cp datos-prueba.sql planfy_postgres:/tmp/seed.sql` && `docker exec planfy_postgres psql -U admin -d planfy -f /tmp/seed.sql`
16. Sembrar también las imágenes de los planes: `docker cp seed-imagenes-safe.sql planfy_postgres:/tmp/img.sql` && `docker exec planfy_postgres psql -U admin -d planfy -f /tmp/img.sql`
17. Acceder al frontend en `http://localhost:4200` y registrar un usuario nuevo o iniciar sesión con las credenciales de prueba (`test@planfy.com` / `test1234`).

#### 4.1.3. Despliegue en el servidor del aula

Para evitar que el tutor tenga que clonar y levantar el proyecto, el código se ha desplegado en el servidor del aula del centro mediante la misma configuración Docker Compose. El acceso desde la red interna del IES Los Alcores se realiza a través de la IP del servidor en los puertos 4200 (frontend) y 8008 (API). Los datos de prueba ya están sembrados, por lo que tras un nuevo registro o el inicio de sesión con la cuenta de prueba se puede empezar a deslizar planes inmediatamente.



```

PS C:\Users\descr\Desktop\Planfy\Planfy> docker compose up -d
[+] Running 3/3
✓ Container planfy_postgres Started 0.4s
✓ Container planfy_backend Started 1.2s
✓ Container planfy_frontend Started 1.8s

PS C:\Users\descr\Desktop\Planfy\Planfy> docker compose ps

```

| NAME            | STATUS     | PORTS                 | IMAGE                  |
|-----------------|------------|-----------------------|------------------------|
| planfy_postgres | Up healthy | 0.0.0.0:5444→5432/tcp | postgres:16            |
| planfy_backend  | Up         | 0.0.0.0:8008→8080/tcp | planfy-backend:latest  |
| planfy_frontend | Up         | 0.0.0.0:4200→80/tcp   | planfy-frontend:latest |

```

PS C:\Users\descr\Desktop\Planfy\Planfy> curl -s http://localhost:8008/actuator/health
{"status":"UP"}

PS C:\Users\descr\Desktop\Planfy\Planfy>

```

Figura 19. Estado de los tres contenedores Docker tras `docker compose up -d`

#### 4.1.4. Variables de entorno y configuración

Los parámetros principales se definen en el fichero `docker-compose.yml`. Para un despliegue en producción se recomienda externalizarlos a un fichero `.env` y modificar los siguientes valores:

- `SPRING_DATASOURCE_PASSWORD`: cambiar `admin_pass` por una contraseña fuerte aleatoria.
- `JWT_SECRET` (en `JwtService.java`): sustituir la cadena de 32 caracteres por una secreta de al menos 256 bits generada con `openssl rand -base64 32`.
- `CORS allowed-origins`: restringir a los dominios reales en lugar de `"*"`.

#### 4.1.5. Detener y limpiar

Para detener los contenedores conservando los datos: `docker compose stop`. Para eliminarlos preservando los volúmenes: `docker compose down`. Para borrar todo, incluida la base de datos: `docker compose down -v`.

## 4.2. Manual de usuario

Planfy ha sido pensada para que cualquier persona pueda utilizarla sin necesidad de formación previa. Los siguientes apartados describen las acciones disponibles desde la perspectiva del usuario final.

### 4.2.1. Registro y acceso

Al acceder por primera vez se presenta una pantalla con dos opciones: iniciar sesión o registrar una cuenta nueva mediante el enlace «¿No tienes cuenta? Regístrate gratis». El registro requiere nombre, correo y contraseña (mínimo seis caracteres). Tras un registro exitoso, el usuario es redirigido automáticamente al panel principal y no necesita iniciar sesión nuevamente.



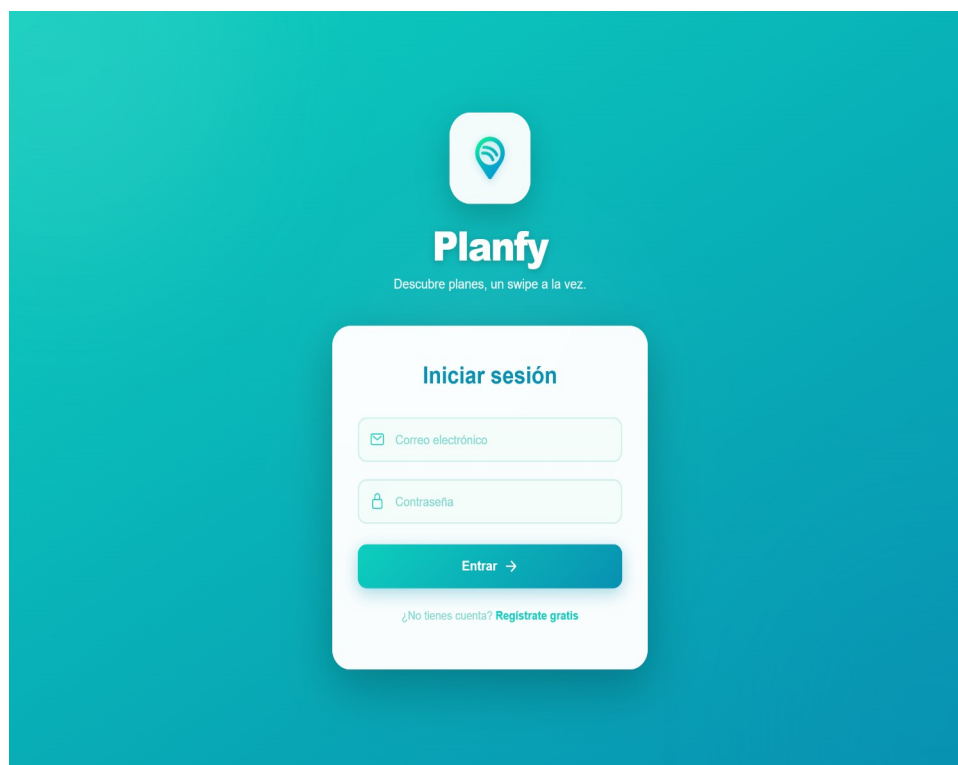


Figura 20. Formulario de inicio de sesión (la cara posterior de la flip-card contiene el formulario de registro)

#### 4.2.2. Descubrimiento de planes (swipe)

La pantalla principal muestra una tarjeta con la información del plan actual. El usuario puede deslizar la tarjeta hacia la derecha para indicar que le gusta o hacia la izquierda para descartarlo; alternatively puede pulsar los botones inferiores con un corazón o una equis. También están disponibles las flechas izquierda y derecha del teclado. El icono de información (i) abre un modal con los detalles ampliados del plan.

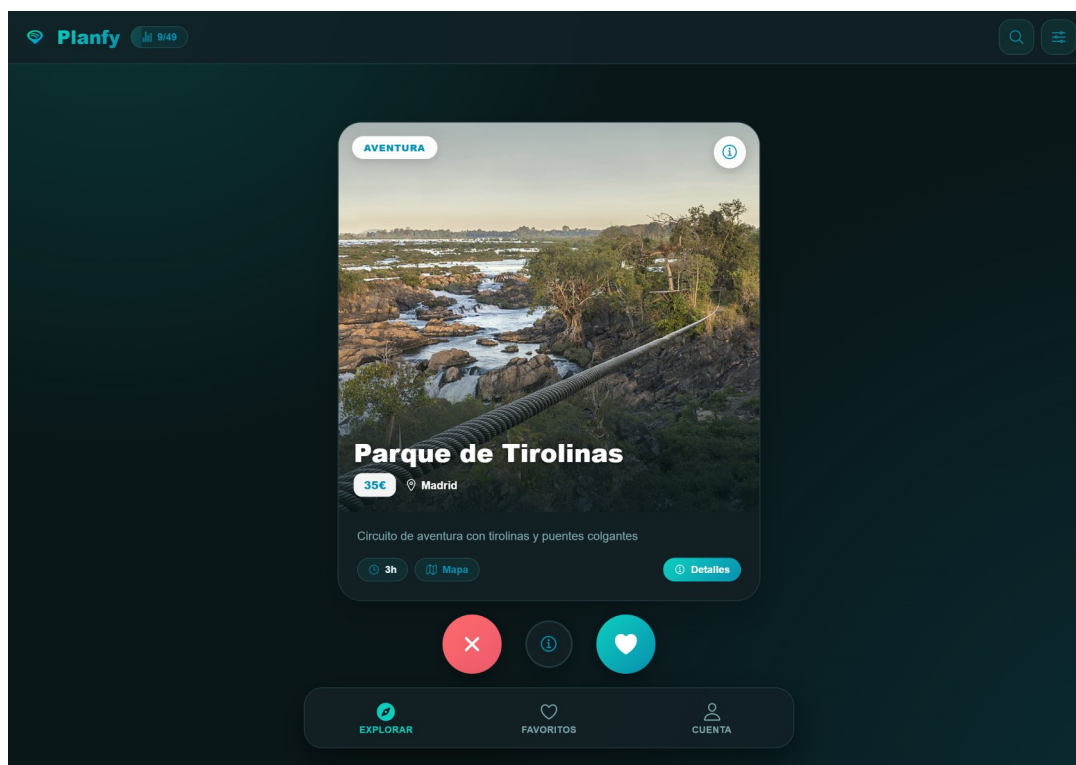


Figura 21. Dashboard en modo oscuro mostrando todos los controles disponibles

### 4.2.3. Filtros y búsqueda

El icono de filtros (deslizadores horizontales) en la esquina superior derecha despliega un panel con tres controles: ciudad, categoría y precio máximo. Aplicar cualquier filtro provoca que la siguiente card mostrada respete las restricciones. La barra de búsqueda (icono de lupa) permite escribir libremente; si el texto coincide con una ciudad o una categoría, se aplica automáticamente como filtro.

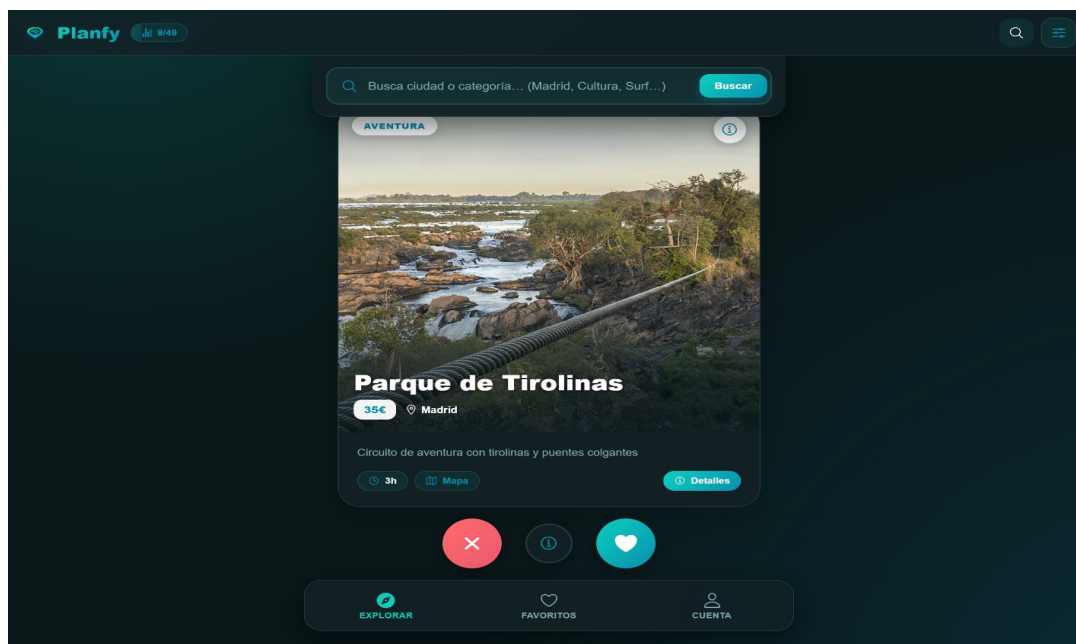


Figura 22. Barra de búsqueda inteligente que detecta automáticamente ciudades y categorías

#### 4.2.4. Favoritos

Todos los planes a los que se les ha dado «me gusta» aparecen automáticamente en la pestaña Favoritos del menú inferior. Esta pantalla muestra una cuadrícula adaptativa (1 columna en móvil, 2 en tablet, 3 en escritorio y hasta 4 en pantallas anchas) con cards que incluyen la imagen, el nombre, el precio, una breve descripción y los chips de duración y ciudad. Para eliminar un favorito basta con pulsar el icono de corazón tachado en la esquina superior derecha de la card.

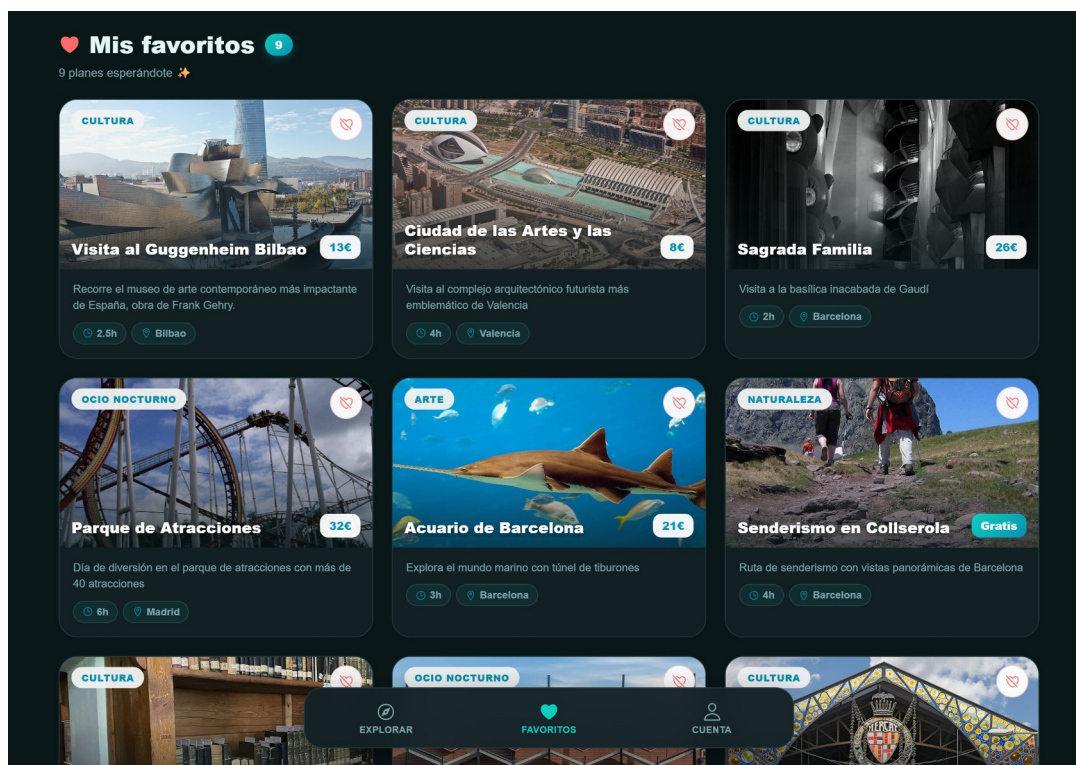


Figura 23. Cuadrícula de favoritos en pantalla amplia

#### 4.2.5. Cuenta y personalización

La pestaña Cuenta presenta el avatar del usuario, su correo electrónico y cuatro tarjetas de estadísticas: Favoritos, Categoría top, Ciudad top y Gratuitos. Las dos centrales son interactivas: al pulsarlas, se vuelve al dashboard con el filtro correspondiente preaplicado. Más abajo se encuentra el selector de tema (Claro / Oscuro / Auto), la información de sesión y el botón Cerrar sesión.

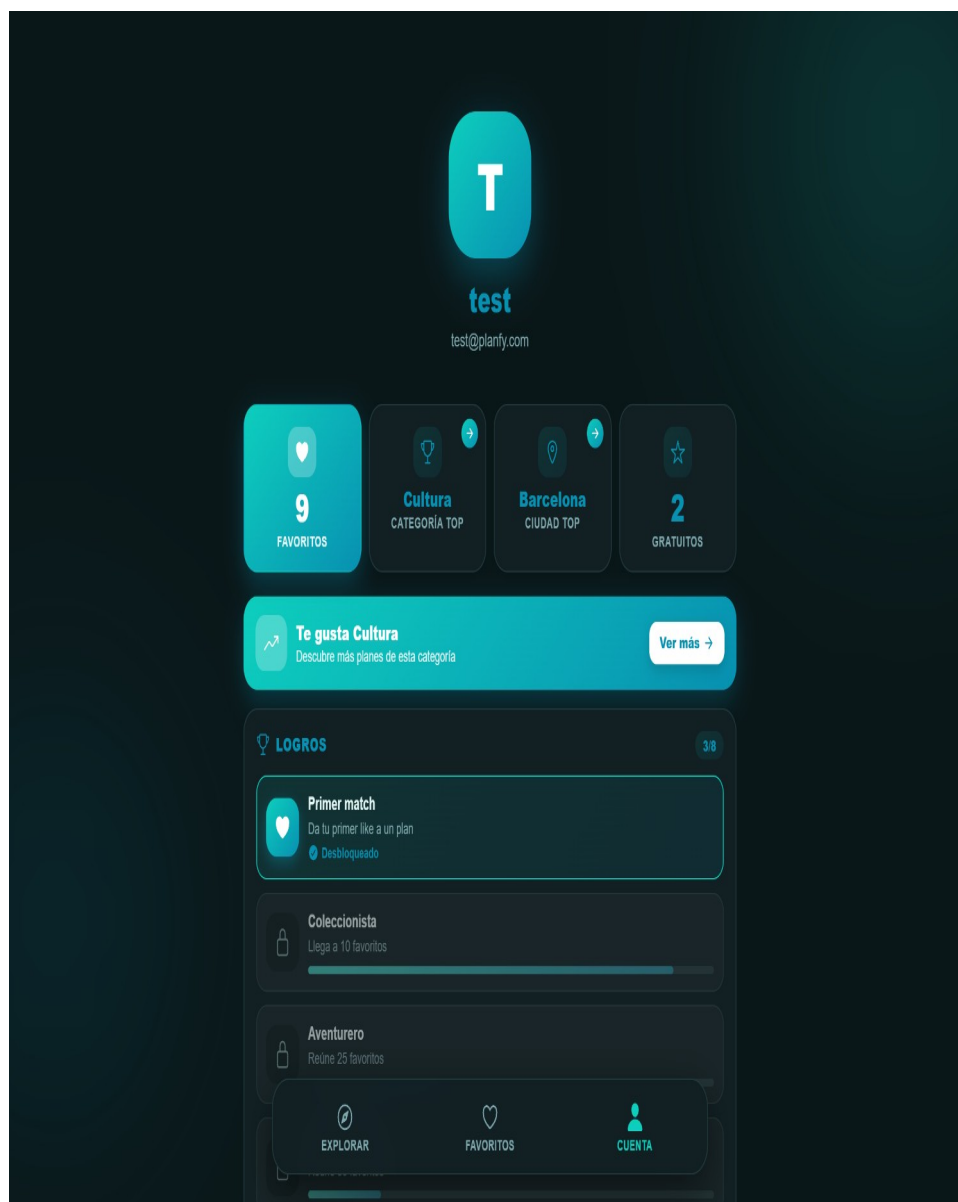


Figura 24. Pantalla completa de cuenta con todas sus secciones

### 4.3. Enlace a la presentación

La presentación de defensa del proyecto, con un mínimo de 15 diapositivas, se encuentra disponible en formato Google Slides en el enlace siguiente:

Presentación:

## 5. Conclusiones finales

### 5.1. Tiempo estimado y tiempo empleado

La planificación inicial estimaba un esfuerzo total de aproximadamente 100 horas distribuidas a lo largo del segundo trimestre. La realidad ha superado ligeramente esa cifra debido principalmente a la fase de pruebas y depuración, donde aparecieron incidencias inesperadas (registro del interceptor en main.ts en lugar de app.config.ts, manejo de excepciones JWT en el filtro, validación rigurosa de URLs de imagen, etcétera). La siguiente tabla compara las estimaciones con los tiempos reales:

| Fase                                            | Estimado (h) | Empleado (h) |
|-------------------------------------------------|--------------|--------------|
| Análisis y diseño                               | 15           | 14           |
| Implementación backend (Spring Boot, BBDD, JWT) | 30           | 38           |
| Implementación frontend (Angular, Ionic, UI)    | 40           | 56           |
| Pruebas, despliegue Docker y depuración         | 10           | 14           |
| Documentación y memoria                         | 15           | 12           |
| <b>TOTAL</b>                                    | <b>110</b>   | <b>134</b>   |

### 5.2. Grado de cumplimiento de los objetivos

La totalidad de los requisitos funcionales formulados en el apartado 3.1 se ha implementado y validado, con la única excepción del CU-11 (panel administrativo de gestión del catálogo) que se planteó como ampliación futura. Los doce requisitos no funcionales también se cumplen: la aplicación responde con tiempos por debajo de los 500 ms para las peticiones de swipe, las contraseñas se almacenan hasheadas con BCrypt, el diseño responsivo funciona desde 320 px hasta 4K, y todo el entorno se levanta con un único comando Docker Compose.

Adicionalmente, durante el desarrollo se han añadido funcionalidades no contempladas inicialmente que enriquecen la experiencia: el sistema de logros, las recomendaciones inteligentes basadas en categoría favorita, el tour de bienvenida y la integración de imágenes temáticas reales procedentes de Wikipedia Commons. La sustitución de las URLs aleatorias originales por imágenes verificadas de fuentes moderadas garantiza la idoneidad del contenido visual.

### 5.3. Propuestas de modificaciones y ampliaciones futuras

A partir de la experiencia adquirida durante el desarrollo y de las observaciones recibidas durante las pruebas, se proponen las siguientes líneas de evolución para una segunda fase del proyecto:

- Implementar el panel administrativo CU-11 para que un usuario con rol ADMIN pueda crear, editar y eliminar planes desde el navegador, sustituyendo el actual seed manual.
- Añadir geolocalización mediante la API de Capacitor Geolocation, de manera que el dashboard ofrezca «Planes a menos de X km de mi posición actual» calculando distancias mediante la fórmula del haversine.

- Incorporar un sistema de reseñas y puntuaciones, con valoración media por plan visible en la card y lista de comentarios de otros usuarios.
- Desarrollar un módulo social que permita seguir a otros usuarios y descubrir planes en común («matches»), inspirado en aplicaciones como Bumble Friends.
- Migrar la autenticación a OAuth 2.0 con proveedores de identidad externos (Google, Apple) para reducir la fricción del registro.
- Habilitar notificaciones push mediante Capacitor Local Notifications para recordar planes guardados o sugerir nuevos planes según los intereses del usuario.
- Empaquetar y publicar la versión Android en la Google Play Store mediante el módulo Capacitor ya configurado.
- Implementar internacionalización (es, en, ca, eu) extrayendo todos los textos a ficheros de traducción.
- Desplegar el backend en un VPS público con HTTPS gestionado por Let's Encrypt y un dominio personalizado.

## 6. Bibliografía y Webgrafía

Las fuentes consultadas durante el desarrollo del proyecto se recogen a continuación siguiendo, en la medida de lo posible, las normas de citación APA 7ª edición. Se distinguen dos categorías: documentación oficial de los frameworks empleados y artículos técnicos o vídeos de apoyo.

### Documentación oficial

Spring. (2025). *Spring Boot Reference Documentation*.

Spring. (2025). *Spring Security Reference*.

Google. (2025). *Angular Documentation*.

Ionic. (2025). *Ionic Framework Documentation*.

Capacitor. (2025). *Capacitor Documentation*.

PostgreSQL Global Development Group. (2025). *PostgreSQL 16 Documentation*.

Docker, Inc. (2025). *Docker Compose Specification*.

JWT. (2024). *Java JWT — JSON Web Token for Java*.

### Recursos didácticos consultados

Mozilla. (2025). *MDN Web Docs — JavaScript & Web APIs*.

Baeldung. (2024). *Spring Security with JWT Authentication*.

Wikimedia Foundation. (2025). *Wikimedia Commons. Banco de imágenes libres con licencia*.

OpenStreetMap Foundation. (2025). *OpenStreetMap*.

### Materiales del ciclo

- Apuntes y prácticas del módulo Bases de Datos (1.º DAW).
- Apuntes y prácticas del módulo Programación (1.º DAW).
- Apuntes y prácticas del módulo Lenguajes de Marcas y Sistemas de Gestión de Información (1.º DAW).
- Apuntes y prácticas del módulo Entornos de Desarrollo (1.º DAW).
- Apuntes y prácticas del módulo Sistemas Informáticos (1.º DAW).
- Apuntes y prácticas del módulo Desarrollo Web en Entorno Servidor (2.º DAW).
- Apuntes y prácticas del módulo Desarrollo Web en Entorno Cliente (2.º DAW).
- Apuntes y prácticas del módulo Diseño de Interfaces Web (2.º DAW).
- Apuntes y prácticas del módulo Despliegue de Aplicaciones Web (2.º DAW).